



File System Dasar & Disk Scheduling

Week 12: Konsep File, Direktori, Inode, dan Disk Scheduling

informatika-itera.github.io/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

Outline

- 1. Konsep Persistence & Abstraksi File**
- 2. Struktur Internal: Inode & Metadata**
- 3. Penjadwalan Disk (Disk Scheduling)**
- 4. Latihan: Perhitungan Disk Scheduling**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Menjelaskan konsep persistence dan urgensinya dalam sistem operasi.
2. Menggambarkan abstraksi file dan direktori beserta analoginya.
3. Memahami apa itu inode dan perannya dalam file system.
4. Menganalisis masalah dan solusi terkait *disk scheduling* (FCFS, SSTF, SCAN, C-SCAN).

Fokus Kelas Ini

Kita fokus pada **konsep Persistence**, **File System**, dan **Disk Scheduling**.

Konsep Persistence & Abstraksi File

Masalah: RAM itu Pelupa

Data hilang saat listrik mati atau program selesai

Masalah Volatilitas

- Memori utama (RAM) bersifat **volatile**.
- Semua data, perhitungan, dan *state* proses hilang saat:
 - Komputer dimatikan/crash.
 - Program (proses) selesai jalan.

Kebutuhan Kita

Kita butuh menyimpan data hasil kerja (dokumen, foto, kode) agar tetap ada esok hari. Kebutuhan ini menuntut ruang penyimpanan yang besar, tahan lama, dan aman.

Analogi: Meja Kerja vs Lemari Arsip

Mengapa kita butuh dua jenis tempat penyimpanan

RAM = Meja Kerja

- Sangat dekat dan cepat dijangkau.
- Kapasitas terbatas.
- Di akhir hari (dimatikan), meja harus dibersihkan total.

Storage = Lemari Arsip Baja

- Sedikit lebih jauh dan lambat.
- Kapasitas luar biasa besar.
- Data **aman dan permanen** meskipun kantor sedang tutup.

Tugas Sistem Operasi

OS bertugas memindahkan data dari "meja kerja" ke dalam "lemari arsip" secara rapi, agar mudah dicari lagi besok.

Konsep Persistence (Ketahanan Data)

Menjaga data tetap hidup melampaui proses yang membuatnya

Apa itu Persistence?

- Kemampuan sistem mempertahankan *state* (data) meskipun listrik terputus.
- Menggunakan *hardware* khusus: *Hard Disk Drive* (HDD) atau *Solid State Drive* (SSD).

Tantangan OS:

- Disk fisik hanyalah deretan panjang sektor-sektor mentah (biasanya 512 bytes).
- Bagaimana cara pengguna membedakan "foto liburan" dengan "tugas OS" di disk yang sama?

Solusi: Abstraksi

OS tidak membiarkan pengguna membaca disk per sektor. OS menciptakan abstraksi yang mudah dipahami: **File** dan **Direktori**.

Abstraksi Pertama: File

Hanya sekumpulan byte dengan nama

Apa itu File?

File adalah abstraksi OS untuk kumpulan data yang saling terkait. Bagi OS, file hanyalah sebuah **array of bytes** linier.

- Setiap file punya **nama tingkat tinggi** (`tugas.pdf`).
- OS yang bertugas memetakan nama file ini ke blok/sektor fisik di dalam disk.

Nomor Inode

Di sistem UNIX/Linux, di bawah layar, setiap file sebenarnya diidentifikasi dengan nomor unik yang disebut **inode number**. OS mengubah nama file menjadi nomor ini.

Mekanisme Interaksi dengan File

Sistem operasi menyediakan API standar untuk membaca/menulis

```
// 1. Buka/buat file
int fd = open("tugas.txt", O_CREAT | O_WRONLY);

// 2. Tulis data
write(fd, "Halo dunia", 10);

// 3. Sinkronisasi (opsional)
fsync(fd);

// 4. Tutup
close(fd);
```

API File

1. `open()`: Minta izin OS, dapat *file descriptor*.
2. `read()` / `write()`: Pindahkan data.
3. `fsync()`: Paksa data turun dari cache RAM langsung ke disk fisik.
4. `close()`: Lepas sumber daya.

Abstraksi Kedua: Direktori

Agar ribuan file tidak berserakan

Apa itu Direktori?

Secara teknis, direktori hanyalah **file khusus** yang berisi daftar pasangan:
(Nama File → Nomor Inode)

- Karena direktori bisa menyimpan direktori lain, terbentuklah struktur **hierarki (*tree*)**.
- Berawal dari satu *root* (/).

Analogi

File adalah "dokumen" lembaran. Direktori adalah "map" (folder) yang mengelompokkan dokumen-dokumen terkait dalam laci lemari arsip.

Checkpoint 1: Persistence & Abstraksi

Cek pemahaman sebelum lanjut

1. Mengapa kita tidak menyimpan semua data secara permanen di RAM saja?
2. Apa bedanya cara pengguna melihat file vs cara OS melihat file?
3. Apa isi dari sebuah direktori pada tingkatan teknis terbawah?

Kunci Jawaban Singkat

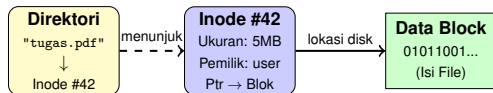
1. RAM bersifat volatile (hilang saat mati) dan kapasitasnya mahal/terbatas.
2. Pengguna melihat nama file (tingkat tinggi), OS melihat file sebagai *array of bytes* yang dipetakan ke blok fisik disk lewat inode.
3. Daftar pemetaan antara "Nama File" ke "Nomor Inode".

Struktur Internal: Inode & Metadata

Apa itu Inode (*Index Node*)?

Pemisahan antara nama, metadata, dan data

Bagi sistem operasi, file **bukanlah** namanya. File direpresentasikan oleh sebuah struktur data bernama **Inode**.



Tiga elemen utama file:

1. **Nama:** Disimpan di Direktori.
2. **Metadata:** Disimpan di Inode.
3. **Isi:** Disimpan di *Data Blocks* (Disk).

Analogi: Perpustakaan Tradisional

Inode seperti kartu katalog

Kartu Katalog = Inode

- Berisi informasi: Judul buku, pengarang, tahun terbit, tebal buku.
- **Lokasi fisik:** Rak A, Baris 2.
- Kartunya sendiri **bukan buku**, ukurannya kecil dan seragam.

Buku Fisik = Data Block

- Buku asli yang tebal dan memakan banyak tempat.
- Ada di rak penyimpanan utama (disk fisik).

Kenapa dipisah?

Sangat efisien mencari kartu katalog yang rapi di laci depan daripada harus membongkar seluruh rak gudang hanya untuk mencari siapa pengarang buku.

Melihat Inode di Terminal

Menggunakan perintah `ls -i` dan `ls -l`

Menampilkan Nomor Inode (`ls -i`)

```
$ ls -i
131093 OS_Slide.pdf
131094 tugas_akhir.docx
```

Menampilkan Metadata Dasar (`ls -l`)

```
$ ls -l OS_Slide.pdf
-rw-r--r-- 1 mario users 1048576 May 5 10:00 OS_Slide
.pdf
```

Apa artinya?

- 131093: Inode number.
- -rw-r--r--: Permissions.
- 1: Jumlah *hard link*.
- mario users: Owner & Group.
- 1048576: Size (dalam bytes).
- May 5: Last modified.

Membedah Inode: Perintah `stat`

Melihat seluruh metadata yang disimpan oleh OS

```
$ stat OS_Slide.pdf
  File: OS_Slide.pdf
  Size: 1048576 Blocks: 2048 IO Block: 4096 regular file
Device: 802h/2050d Inode: 131093 Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  mario)  Gid: ( 100/  users)
Access: 2026-05-05 10:00:00.000000000
Modify: 2026-05-04 15:30:00.000000000
Change: 2026-05-04 15:30:00.000000000
```

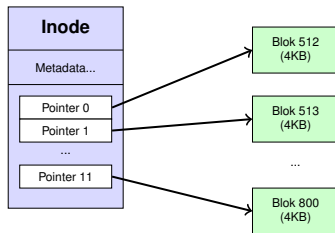
3 Jenis Timestamps

- **Access (atime)**: Kapan file terakhir **dibaca/dibuka**.
- **Modify (mtime)**: Kapan **isi data** file terakhir diubah.
- **Change (ctime)**: Kapan **metadata** terakhir diubah.

Bagaimana Inode Menemukan Data?

Menggunakan Direct Pointers

Setiap file menempati beberapa blok di disk (misal: blok ukuran 4KB). Inode menyimpan daftar nomor blok tersebut (*pointers*).



Direct Pointers

Secara standar, Inode punya 12 pointer langsung. Dengan blok 4KB, ukuran file maksimal hanya: $12 \times 4KB = 48KB$. Bagaimana jika file lebih besar?

Menangani File Besar

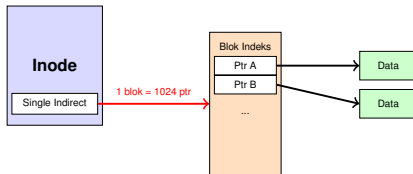
Single, Double, dan Triple Indirect Pointers

Solusi: Blok Indeks

OS menggunakan **Indirect Pointer**: pointer di inode **tidak** menunjuk ke data, melainkan menunjuk ke sebuah blok penuh berisi pointer lain.

- **Single Indirect**: Menunjuk ke blok yang berisi pointer data.
- **Double Indirect**: Menunjuk ke blok yang berisi pointer ke blok *single indirect*.

Dengan *double/triple indirect*, ukuran file bisa mencapai ukuran Terabytes.

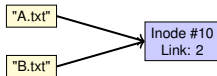


Hard Link vs Soft (Symbolic) Link

Dua cara memberikan nama ganda pada sebuah file

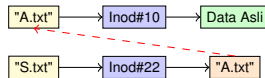
Hard Link (`ln`)

- Membuat **nama baru** di direktori yang menunjuk ke **Inode yang sama**.
- Jika file asli dihapus, datanya tetap ada selama masih ada nama lain yang menunjuk (Link count > 0).



Soft Link (`ln -s`)

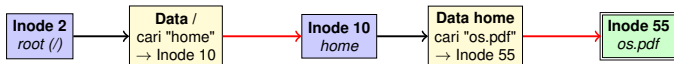
- Memiliki **Inode sendiri**.
- Berisi teks berupa *path* string ke file asli (seperti *shortcut* di Windows).
- Jika file asli dihapus, link ini menjadi "mati" (*dangling link*).



Path Traversal

Cara OS mencari `/home/os.pdf`

Untuk membuka sebuah file, OS harus membaca rentetan direktori dan inode secara bergantian. OS selalu mulai dari **Root Inode** (biasanya bernomor 2).



Mengapa Membaca File Lambat?

Bisa dilihat, sekadar mencari letak `'os.pdf'` membutuhkan **banyak operasi I/O ke disk** (baca inode → baca blok → baca inode → dst). Oleh karena itu, OS menggunakan struktur cache di RAM.

Checkpoint 2: Inode & Metadata

Cek pemahaman sebelum lanjut

1. Apa fungsi utama dari Inode?
2. Perintah Unix apa yang digunakan untuk melihat metadata lengkap (termasuk timestamp) sebuah file?
3. Jika sebuah file 1 GB menggunakan sistem blok 4KB dan 12 *direct pointers*, bagaimana OS menemukan sisa datanya?
4. Apa bedanya jika file "Asli.txt" dihapus: ketika kita membuat *Hard Link* vs *Soft Link*?

Kunci Jawaban

1. Inode menyimpan metadata file (owner, size, permission, timestamps) dan pointer ke blok fisik disk.
2. `stat [nama_file]`.
3. OS menggunakan *Indirect Pointers* (blok yang isinya pointer, bukan data).
4. Hard link: file tetap ada, bisa dibaca (link count > 1). Soft link: *broken/dangling link*, file tidak bisa dibaca.

Penjadwalan Disk (Disk Scheduling)

Mengapa Butuh Penjadwalan Disk?

Masalah utama pada komponen mekanis

I/O Disk Sangat Lambat

Tidak seperti RAM yang elektronis, HDD (Hard Disk Drive) memiliki komponen **mekanis** yang bergerak fisik. CPU bisa mengeksekusi jutaan instruksi dalam waktu yang dibutuhkan HDD untuk membaca satu blok data.

Tujuan Penjadwalan:

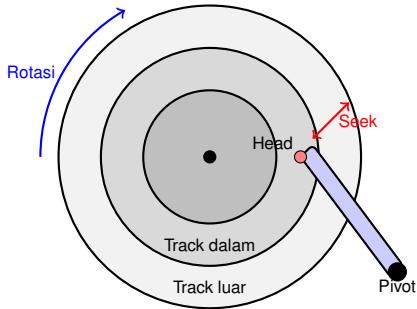
- Mengurangi total waktu pergerakan fisik disk.
- Memaksimalkan *throughput* (jumlah request I/O per detik).

Analogi: Kurir Gudang

Bayangkan kurir di gudang raksasa. Jika dia mengambil paket berdasarkan urutan pesanan masuk tanpa memikirkan rute (loncat dari rak ujung kiri ke ujung kanan terus menerus), dia akan kelelahan dan lambat. **Rute harus dioptimasi.**

Anatomi Hard Disk & Komponen Waktu

Dari mana kelambatan itu berasal?



Dua Faktor Penentu Waktu

1. **Seek Time**: Waktu yang dibutuhkan lengan (*actuator arm*) untuk memindahkan *head* ke *track/silinder* yang tepat. (*Sangat lambat*)
2. **Rotational Latency**: Waktu tunggu hingga *sektor* yang dicari berputar tepat di bawah *head*. (*Cukup lambat*)

Catatan: *Disk Scheduling modern fokus meminimalkan **Seek Time**.*

1. FCFS (First-Come, First-Serve)

Paling sederhana tapi paling boros pergerakan

Konsep FCFS

Permintaan dilayani persis sesuai urutan kedatangannya.

Karakteristik:

- **Adil:** Tidak ada request yang kelaparan (*starvation*).
- **Tidak efisien:** *Head* disk bisa bergerak liar bolak-balik dari ujung ke ujung.

Contoh Kasus

Antrean request track: 98, 183, 37, 122, 14, 124, 65, 67

Posisi Head Awal: **53**

Pergerakan head:

53 → 98 → 183 → 37 → 122 → 14 → 124
→ 65 → 67

Total Pergerakan: 640 silinder!

2. SSTF (Shortest Seek Time First)

Pilih yang paling dekat dari posisi sekarang

Konsep SSTF

Melayani permintaan yang posisinya **paling dekat** dengan posisi *head* saat ini, mengabaikan urutan kedatangan.

Karakteristik:

- **Efisien**: Mengurangi total pergerakan head secara drastis dibanding FCFS.
- **Masalah**: Rentan **Starvation**. Request yang jauh bisa tidak dilayani terus menerus jika request baru bermunculan di dekat head.

Contoh Kasus

Antrean: 98, 183, 37, 122, 14, 124, 65, 67

Posisi Head Awal: **53**

Pergerakan head (cari terdekat):

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Total Pergerakan: 236 silinder! (Jauh lebih baik dari 640)

3. Algoritma SCAN (Elevator Algorithm)

Maju terus pantang mundur (sampai mentok)

Konsep SCAN

Head bergerak ke satu arah (misal ke track dalam), melayani semua request di jalurnya. Saat sampai di **ujung disk**, ia berbalik arah dan melayani sisa request di jalur pulang.

Analogi: Lift

Lift naik melayani semua lantai yang ditekan sampai lantai teratas, baru setelah itu turun melayani lantai bawah.

Karakteristik:

- Mencegah *starvation* ekstrem dari SSTF.
- Efisien seperti arah sapuan.
- **Masalah:** Kurang adil bagi request yang berada tepat di belakang arah pergerakan *head*, karena harus menunggu *head* sampai ujung dan berbalik arah.

4. Algoritma C-SCAN (Circular SCAN)

Sapuan satu arah demi keadilan

Konsep C-SCAN

Sama seperti SCAN, namun ketika *head* mencapai ujung disk, ia **langsung melompat kembali ke titik awal** ujung lainnya **tanpa melayani request** di jalan pulang.

Mengapa C-SCAN lebih baik dari SCAN?

- Pada SCAN biasa, ketika head berbalik dari ujung, area tersebut baru saja disapu (kemungkinan request barunya sedikit).
- Area di ujung lain sudah menunggu **sangat lama**.



- **Kelebihan:** Memberikan waktu tunggu yang jauh lebih seragam bagi semua request.

5. LOOK dan C-LOOK

Varian pintar dari SCAN

Masalah SCAN/C-SCAN Murni

Mereka selalu bergerak **sampai ujung disk (track 0 atau ujung maksimal)**, bahkan jika tidak ada lagi request di arah tersebut. Ini membuang waktu.

Algoritma LOOK:

- Varian SCAN. Head hanya bergerak sejauh **request terakhir** di arah tersebut, lalu berbalik.

Algoritma C-LOOK:

- Varian C-SCAN. Head menyapu satu arah, berhenti di request terjauh, lalu **melompat balik ke request terendah** (bukan track 0).
- Ini adalah algoritma yang **paling sering diimplementasikan** di dunia nyata untuk HDD mekanis.

Ringkasan Perbandingan Algoritma

Mana yang terbaik?

Algoritma	Kelebihan Utama	Kelemahan Utama
FCFS	Adil, sangat mudah diimplementasikan.	Sangat lambat, pergerakan head boros.
SSTF	Kinerja sangat baik untuk batch request.	Resiko <i>starvation</i> tinggi bagi track jauh.
SCAN	Kinerja baik, mencegah <i>starvation</i> .	Tidak adil bagi area yang baru saja dilewati head.
C-SCAN	Waktu tunggu sangat seragam/adil.	Head tetap pergi ke ujung disk membuang waktu.
C-LOOK	Efisien, seragam, tidak ke ujung disk.	Overhead komputasi mencari letak request ujung.

Catatan untuk SSD

SSD (Solid State Drive) **tidak memiliki piringan berputar** atau lengan mekanis (tidak ada *seek time*). OS biasanya menggunakan FCFS yang dimodifikasi, atau membiarkan *firmware* SSD mengatur penjadwalannya sendiri.

Checkpoint 3: Disk Scheduling

Cek pemahaman sebelum latihan

1. Apa dua komponen utama yang membuat operasi disk I/O lambat?
2. Mengapa algoritma SSTF bisa menyebabkan sebuah proses menunggu selamanya (*starvation*)?
3. Apa bedanya algoritma SCAN biasa dengan LOOK?
4. Mengapa algoritma penjadwalan pergerakan head (seperti C-SCAN) menjadi kurang relevan pada teknologi penyimpanan modern saat ini?

Kunci Jawaban Singkat

1. *Seek time* (pergerakan lengan) dan *Rotational latency* (putaran piringan).
2. Jika request baru selalu berdatangan di area dekat head, request di ujung disk tidak akan pernah dipilih.
3. SCAN berjalan mentok sampai ujung disk. LOOK hanya berjalan sejauh posisi request terjauh lalu berbalik.
4. Karena penggunaan **SSD (Solid State Drive)** yang murni elektronik tanpa lengan penggerak/piringan, sehingga tidak ada konsep fisik *seek time*.

Latihan: Perhitungan Disk Scheduling

Soal Latihan

Hitung total pergerakan head (dalam silinder)

Skenario

Sebuah disk drive memiliki **200 silinder**, dinomori dari **0 hingga 199**. Posisi *head* disk saat ini berada di silinder **53**.

Antrean permintaan akses I/O (*queue*) datang dengan urutan berikut:

98, 183, 37, 122, 14, 124, 65, 67

Tugas: Hitung **total jarak pergerakan head** untuk algoritma berikut:

1. **FCFS** (First-Come, First-Serve)
2. **SSTF** (Shortest Seek Time First)
3. **SCAN** (Asumsi arah awal menuju silinder lebih besar / 199)
4. **C-SCAN** (Asumsi arah awal menuju silinder lebih besar / 199)

Jawaban: FCFS

Sesuai urutan kedatangan

Antrean: 98, 183, 37, 122, 14, 124, 65, 67

Posisi Awal: 53

Jalur Pergerakan: $53 \rightarrow 98 \rightarrow 183 \rightarrow 37 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$

Perhitungan Jarak:

- $|53 - 98| = 45$
- $|98 - 183| = 85$
- $|183 - 37| = 146$
- $|37 - 122| = 85$
- $|122 - 14| = 108$

Lanjutan:

- $|14 - 124| = 110$
- $|124 - 65| = 59$
- $|65 - 67| = 2$

Total Pergerakan

640 silinder

Jawaban: SSTF

Pilih yang terdekat dari posisi sekarang

Antrean (Sorted): 14, 37, 65, 67, 98, 122, 124, 183

Posisi Awal: 53

Jalur Pergerakan: 53 $\rightarrow$ 65 $\rightarrow$ 67 $\rightarrow$ 37 $\rightarrow$ 14 $\rightarrow$ 98 $\rightarrow$ 122 $\rightarrow$ 124 $\rightarrow$ 183

Perhitungan Jarak:

- $|53 - 65| = 12$
- $|65 - 67| = 2$
- $|67 - 37| = 30$
- $|37 - 14| = 23$
- $|14 - 98| = 84$

Lanjutan:

- $|98 - 122| = 24$
- $|122 - 124| = 2$
- $|124 - 183| = 59$

Total Pergerakan

236 silinder

Jawaban: SCAN

Maju ke ujung (199), lalu berbalik arah

Antrean (Sorted): 14, 37, 65, 67, 98, 122, 124, 183

Posisi Awal: 53 **Arah:** Ke atas (menuju 199)

Jalur Pergerakan: 53 → 65 → 67 → 98 → 122 → 124 → 183 → **199** → 37 → 14

Perhitungan Jarak:

- Perjalanan naik (53 ke 199):
 $|199 - 53| = 146$
- Perjalanan turun (199 ke 14):
 $|199 - 14| = 185$

Cara Cepat

Tujuan terjauh ke kanan: 199

Tujuan terjauh ke kiri: 14

Total = $(199 - 53) + (199 - 14)$

Total = $146 + 185 = \mathbf{331 \text{ silinder}}$

Jawaban: C-SCAN

Maju ke ujung (199), lompat ke awal (0), maju lagi

Antrean (Sorted): 14, 37, 65, 67, 98, 122, 124, 183

Posisi Awal: 53 **Arah:** Ke atas (menuju 199)

Jalur Pergerakan: 53 $\rightarrow$... $\rightarrow$ 183 $\rightarrow$ **199** $\rightarrow$ **0** $\rightarrow$ 14 $\rightarrow$ 37

Perhitungan Jarak:

- Perjalanan naik (53 ke 199):
 $|199 - 53| = 146$
- Lompat ke awal (199 ke 0):
 $|199 - 0| = 199$
- Lanjut naik (0 ke 37):
 $|37 - 0| = 37$

Total Pergerakan

Total = 146 + 199 + 37
= 382 silinder

Catatan: Beberapa literatur mengabaikan jarak lompatan (return time), namun dalam konteks OS umumnya jarak lompat silinder ini dihitung sebagai total seek movement.

Terima Kasih

Ada Pertanyaan?

Sampai jumpa di pertemuan minggu depan!